Intro to RDBMS

Database Management System (DBMS)

- DBMS contains information about a particular enterprise
  - Collection of interrelated data
  - Set of programs to access the data
  - An environment that is both *convenient* and *efficient* to use
- Database applications:
  - Banking: transactions
  - Airlines: reservations, schedules
  - Universities: registration, grades

Database Management System (DBMS) (cont.)

- Database applications (cont.)
  - Sales: customers, products, purchases
  - Online retailers: order tracking, customized recommendations
  - Manufacturing: production, inventory, orders, supply chain
  - Human resources: employee records, salaries, tax deductions
- Databases can be very large
- Databases touch all aspects of our lives

University Database Example

- Application program examples
  - Add new students, instructors, and courses
  - Register students for courses, and generate class rosters
  - Assign grades to students, compute grade point averages (GPA) and generate transcripts
- In the early days, database applications were built directly on top of file systems

Levels of Abstraction

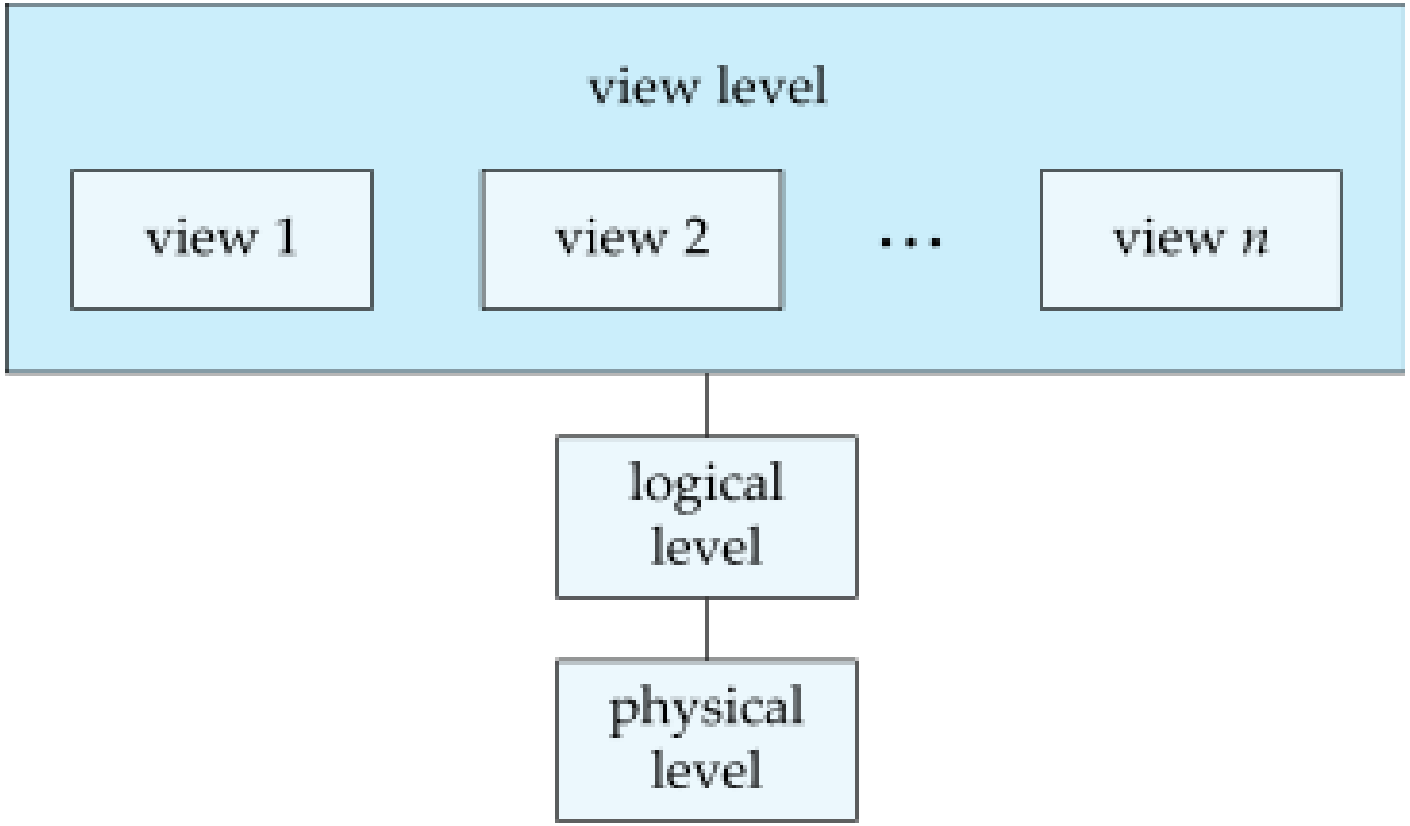
- ***Physical level:*** describes how a record (for example, customer) is stored
- ***Logical level:*** describes data stored in database, and the relationships among the data

```
type instructor = record
                        ID : string;
name : string;
dept_name : string;
salary : integer;
end;
```

- ***View level:*** application programs hide details of data types; views can also hide information (such as an employee's salary) for security purposes

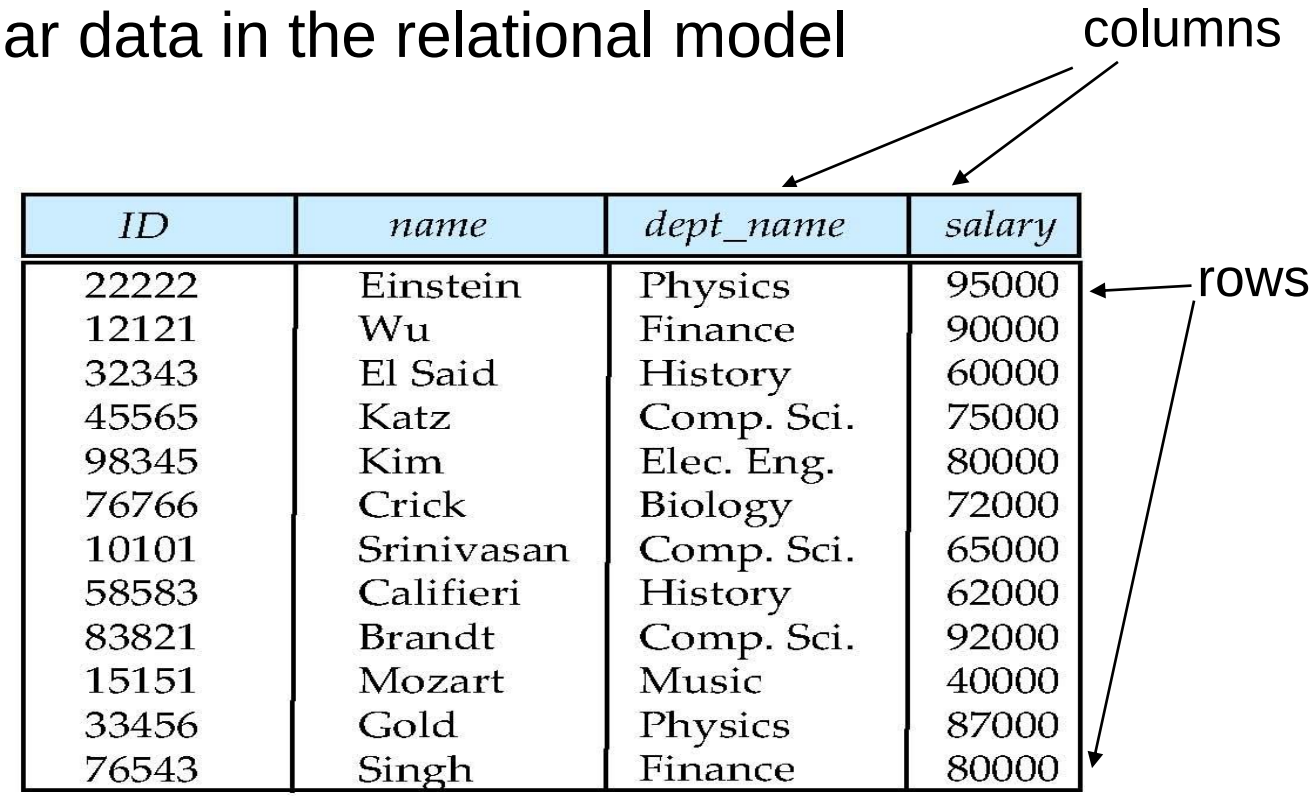
View of Data

An architecture for a database system



Relational Model

- Relational model
- Example of tabular data in the relational model



ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

A Sample Relational Database

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Few Definitions

- Entity – An object, concept or thing about which data is stored.
  - For example -- In University database Student, Professors, Courses, etc. are all examples for Entity.
- Attributes: Qualities associated with the entity.
  - Each Entity has attributes
  - For example – Dept Name, Department ID are examples of attributes within Department.
- Entity Set: A set of related entities
- Relations – A two-dimensional representation of entities and relationship
- Relationship – An inherent mapping involving two or more entities.
- Tuples – Correspond to rows in a table

DataScience@SMU

How Did We Get Here

History of Database Systems

- 1950s and early 1960s
  - Data processing using magnetic tapes for storage
    - Tapes provided only sequential access
  - Punched cards for input
- Late 1960s and 1970s
  - Hard disks allowed direct access to data
  - Network and hierarchical data models in widespread use
- Ted Codd defines the relational data model
  - Would win the ACM Turing Award for this work
  - IBM Research begins System R prototype
  - UC Berkeley begins ingres prototype
- High performance (for the era) transaction processing

History of Database Systems (cont.)

- 1980s
  - Research relational prototypes evolve into commercial systems
    - SQL becomes industrial standard
  - Parallel and distributed database systems
  - Object-oriented database systems
- 1990s
  - Large decision support and data-mining applications
  - Large multi-terabyte data warehouses
  - Emergence of web commerce
- Early 2000s
  - XML and XQuery standards
  - Automated database administration
- Later 2000s
  - Giant data storage systems
    - Google BigTable, Yahoo PNuts, Amazon, ...

Drawbacks of Using File Systems to Store Data, Part I

- Data redundancy and inconsistency
  - Multiple file formats, duplication of information in different files
- Difficulty in accessing data
  - Need to write a new program to carry out each new task
- Data isolation: multiple files and formats
- Integrity problems
  - Integrity constraints (for example, account balance greater than zero) become “buried” in program code rather than being stated explicitly
  - Hard to add new constraints or change existing ones

Drawbacks of Using File Systems to Store Data, Part II

- Atomicity of updates
  - Failures may leave database in an inconsistent state with partial updates carried out
  - Example: transfer of funds from one account to another should either complete or not happen at all
- Concurrent access by multiple users
  - Concurrent access needed for performance
  - Uncontrolled concurrent accesses can lead to inconsistencies
    - Example: two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time

Drawbacks of Using File Systems to Store Data, Part III

- Security problems
  - Hard to provide user access to some, but not all, data
- ***Database systems offer solutions to all the above problems***

DataScience@SMU

Why RDBMS

Database Architecture

- The architecture of a database systems is greatly influenced by the underlying computer system on which the database is running:
  - Centralized
  - Client-server
  - Parallel (multi-processor)
  - Distributed

Levels of Abstraction

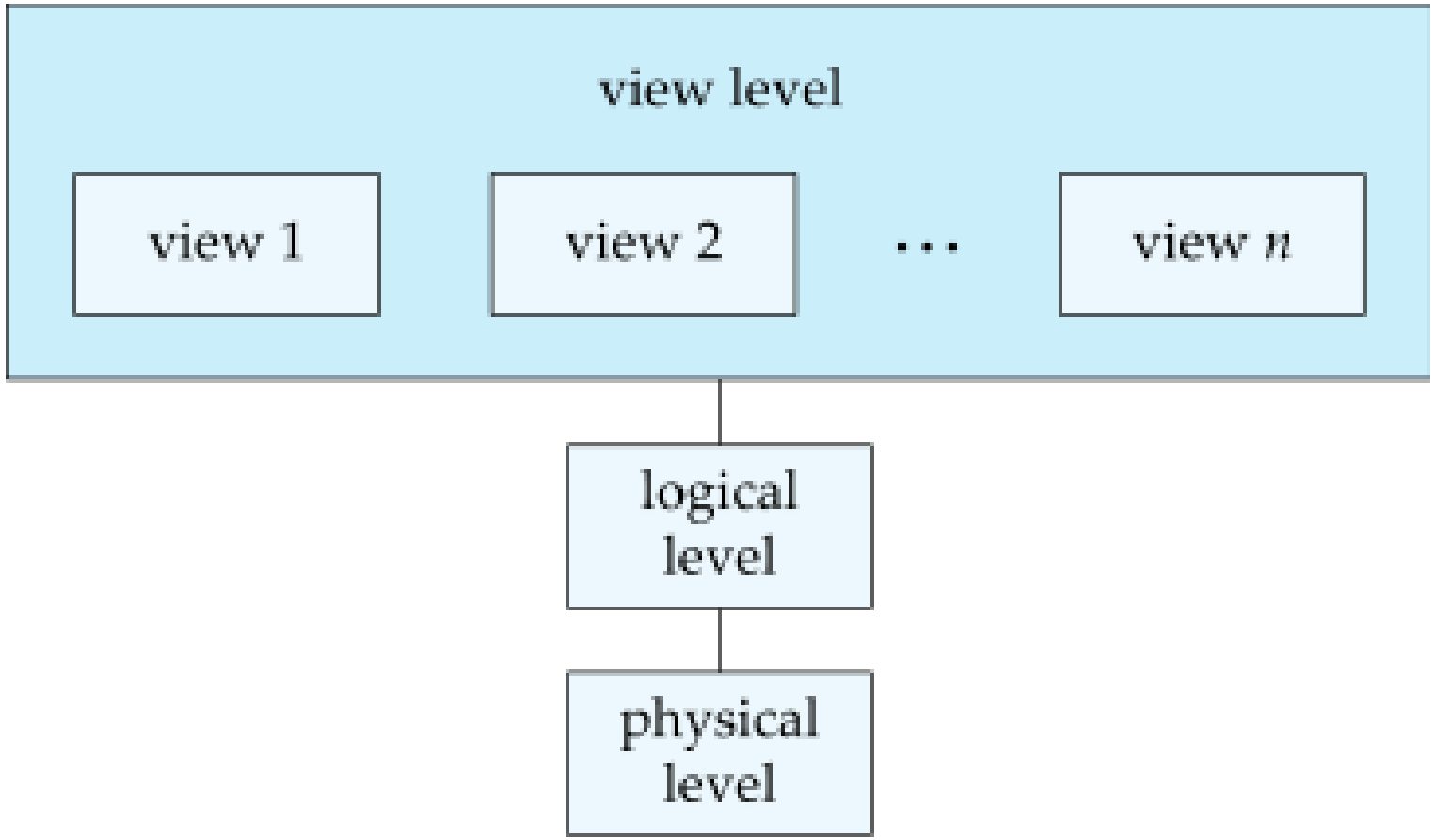
- ***Physical level*:** describes how a record (for example, customer) is stored
- ***Logical level:*** describes data stored in database, and the relationships among the data

```
type instructor = record
                        ID : string;
name : string;
dept_name : string;
salary : integer;
end;
```

- ***View level:*** application programs hide details of data types; views can also hide information (such as an employee's salary) for security purposes

View of Data

An architecture for a database system



Instances and Schemas

- Similar to types and variables in programming languages
- ***Schema:*** the logical structure of the database
  - Example: the database consists of information about a set of customers and accounts and the relationship between them
  - Analogous to type information of a variable in a program
  - **Physical schema:** database design at the physical level
  - **Logical schema:** database design at the logical level
- ***Instance:*** the actual content of the database at a particular point in time
  - Analogous to the value of a variable

Instances and Schemas (cont.)

- ***Physical data independence:*** the ability to modify the physical schema without changing the logical schema
  - Applications depend on the logical schema
  - In general, the interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others

SQL

- ***SQL****:* widely used non-procedural language
  - Example: find the name of the instructor with ID 22222

    select *name*
    from *instructor*
    where *instructor.ID* = ‘22222’
  - Example: find the ID and building of instructors in the physics department

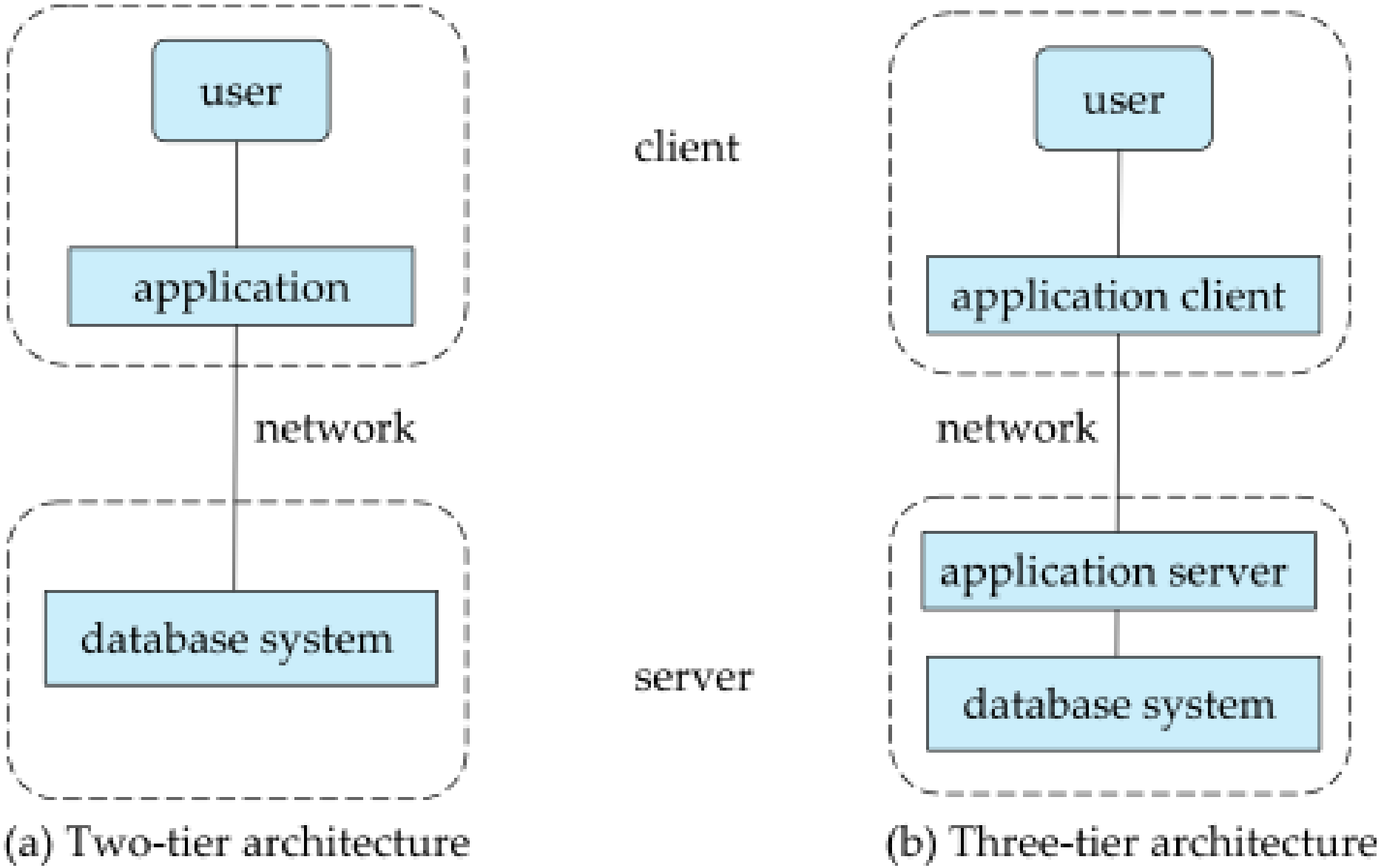
select *instructor*.*ID*, *department*.*building*
from *instructor*, *department*
where *instructor.dept_name = department.dept_name* **and**
department.dept_name = ‘Physics’

SQL (cont.)

- Application programs generally access databases through one of
  - Language extensions to allow embedded SQL
  - Application program interface (for example, ODBC/JDBC) which allow SQL queries to be sent to a database
- Chapters three, four and five

DataScience@SMU

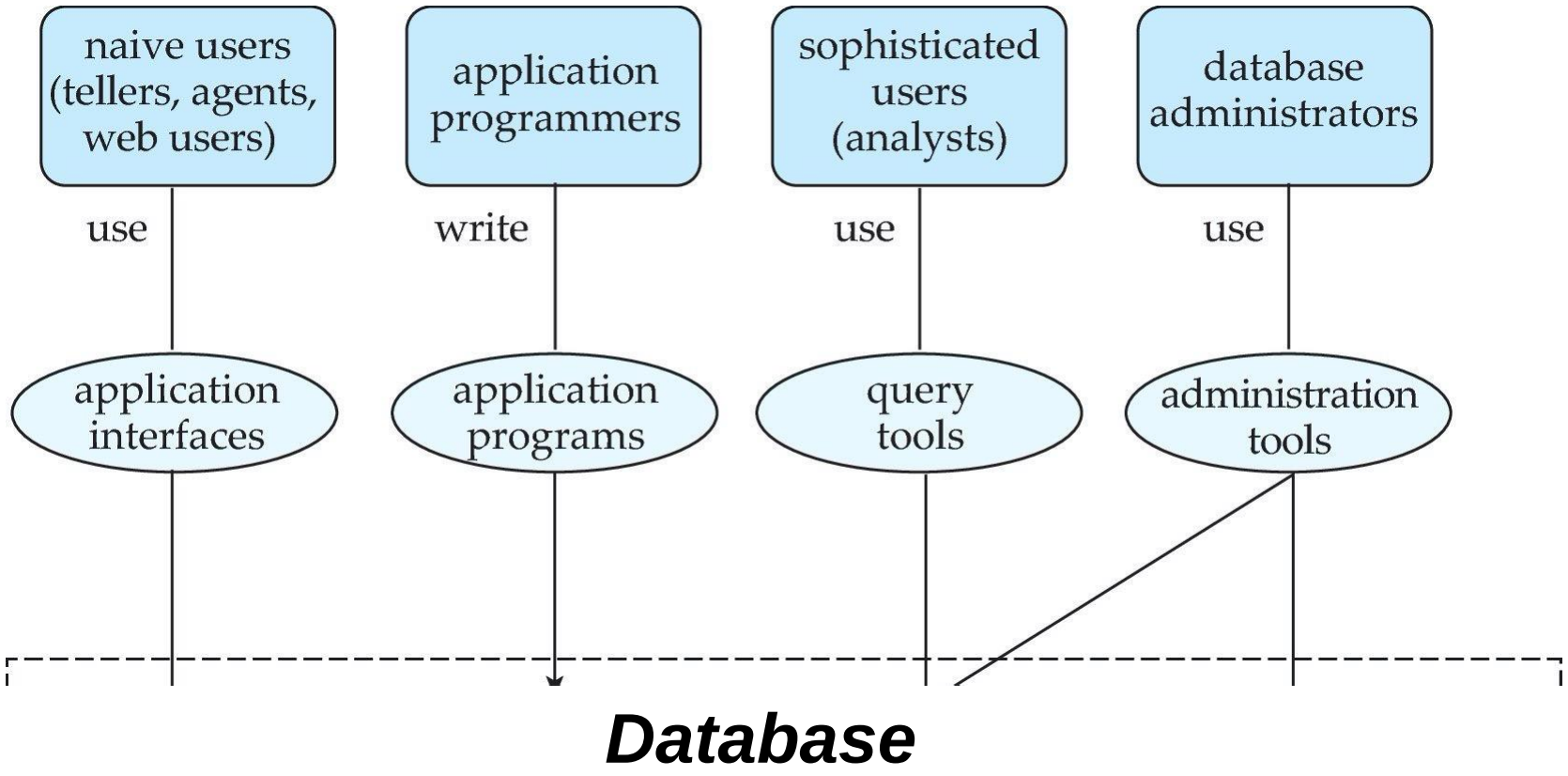
Actors and Their Functions



(a) Two-tier architecture

(b) Three-tier architecture

Database Users and Administrators



DB Administrator

- Responsible for:
  - Installing DBs
  - DB Configuration
  - Overall administration
  - Ensuring everything is up and running
    - Monitoring

DB Architect

- Responsible for:
  - Design and development
  - Translate logical data models into physical database design
  - Overall administration
  - Ensuring everything is up and running
    - Monitoring

Others

- DB modeler
- Data warehouse administrator
- Performance analysts

DataScience@SMU

Oracle & MSSQL

Oracle & MSSQL

- Two of the widely used Databases along with DB2
- Oracle is used by large Enterprises for their mission critical applications
- Oracle used Maximum Availability Architecture
- Oracle uses PL/SQL (Procedural Language/SQL)
- MSSQL

Oracle & MSSQL

- Used by many organizations
- Similar to Oracle DB
- MSSQL uses T-SQL (Transact SQL)

PL/SQL – T-SQL

- Two languages handle variables, Stored Procedures and built-in functions differently.
- PL/SQL can also group procedures together into packages,
- PL/SQL more powerful yet complex
- T-SQL is much more simple and easier to use.

DataScience@SMU

MySQL Overview

MySQL

- Open source database
  - Free to use
- Provides interactive shell for operating on Database
- Can also use any GUI based tool to connect to MySQL instance.

MySQL

- Once logged in, you can try some simple queries.
- For example:

```
mysql> SELECT VERSION(), CURRENT_DATE;
```

- Note that most MySQL commands end with a semicolon (;)
- MySQL returns the total number of rows found, and the total time to execute the query.

MySQL

- Mysql determines where your statement ends by looking for the terminating semicolon, not by looking for the end of the input line.

- Here's a simple multiple-line statement:

```
mysql> SELECT
 -> USER()
 -> ,
 -> CURRENT_DATE;
```

MySQL

- To get started on your own database, first check which databases currently exist.
- Use the SHOW statement to find out which databases currently exist on the server:

```
mysql> show databases;
```

MySQL

- To create a new database, issue the “create database” command:
  - **`mysql> create database webdb;`**
- To the select a database, issue the “use” command:
  - **`mysql> use webdb;`**

DataScience@SMU